

## □ Milestone 2 —Enrichissement du modèle

**Sprint V2 ·Al Qalam Stock Manager** Thème : méthodes magiques  
·gestionnaire de contexte ·tri ·dialogues

---

### Objectif

Enrichir les classes V1 avec les **méthodes spéciales de Python** (`__eq__`, `__lt__`, `__len__`, `__contains__`, `__iter__`, `__enter__`, `__exit__`) pour rendre les objets comparables, itérables et triables nativement, et ajouter le tri par colonne, la fiche détail et la modification de produit.

---

### Ce qui change par rapport à V1

```
alqalamv2/
├── models/
│   └── produit.py    ← + __eq__ __lt__ __hash__ __str__ __repr__
├── services/
│   └── stock_service.py ← + FichierStock (context manager)
│                           + __len__ __contains__ __iter__ __str__
│                           + mettre_a_jour_produit() supprimer_produit()
├── ui/frames/
│   └── stock_frame.py    ← + tri par colonne, double-
clic → fiche détail
└── dialogs.py          ← + DialogueFicheDetail + DialogueModification
```

---

### Concepts mis en œuvre

| Méthode magique                  | Opérateur / usage                      | Bénéfice                       |
|----------------------------------|----------------------------------------|--------------------------------|
| <code>__eq__(self, other)</code> | <code>produit_a == produit_b</code>    | Comparaison par référence      |
| <code>__lt__(self, other)</code> | <code>produit_a &lt; produit_b</code>  | Tri avec <code>sorted()</code> |
| <code>__hash__(self)</code>      | <code>set(produits)</code> , dict keys | Stockage dans ensemble/dict    |
| <code>__str__(self)</code>       | <code>print(produit)</code>            | Affichage lisible              |
| <code>__repr__(self)</code>      | En débogueur                           | Représentation technique       |

| Méthode magique                      | Opérateur / usage                    | Bénéfice                 |
|--------------------------------------|--------------------------------------|--------------------------|
| <code>__len__(self)</code>           | <code>len(stock)</code>              | Nombre de produits       |
| <code>__contains__(self, ref)</code> | <code>"CRAY-001" in stock</code>     | Test d'appartenance      |
| <code>__iter__(self)</code>          | <code>for p in stock:</code>         | Itération sur le stock   |
| <code>__enter__ / __exit__</code>    | <code>with FichierStock(...):</code> | Gestionnaire de contexte |

## Guide de réalisation étape par étape

### Étape 1 —Méthodes magiques sur Produit (15 min)

*# models/produit.py — ajouts V2*

```
class Produit:
    # ... (tout le code V1 conservé) ...

    # — Comparaison —————
    def __eq__(self, other) -> bool:
        """Deux produits sont égaux si leur référence est identique."""
        if not isinstance(other, Produit):
            return NotImplemented
        return self.ref == other.ref

    def __lt__(self, other) -> bool:
        """Tri naturel : par référence alphabétique."""
        if not isinstance(other, Produit):
            return NotImplemented
        return self.ref < other.ref

    def __hash__(self) -> int:
        """Permet d'utiliser Produit dans un set ou comme clé de dict."""
        return hash(self.ref)

    # — Représentation —————
    def __str__(self) -> str:
        statut = "⚠ ALERTE" if self.est_en_alerte() else "✅ OK"
        return f"[{self.ref}] {self.nom} — Qté: {self.qte} / Seuil: {self.seuil_min}"

    def __repr__(self) -> str:
```

```

    return (f"Produit(ref={self.ref!r}, nom={self.nom!r}, "
            f"qte={self.qte}, seuil_min={self.seuil_min})")

```

### Pourquoi définir `__hash__` quand on définit `__eq__` ?

Python exige que si deux objets sont égaux (`a == b`), ils aient le même hash (`hash(a) == hash(b)`). Si vous définissez `__eq__` sans `__hash__`, Python rend l'objet non-hashable (ne peut plus être mis dans un set ou utilisé comme clé de dict).

*# Démonstration*

```

p1 = Produit("CRAY-001", "Crayon HB", "Écriture", 0.15, 0.50, 150, 20)
p2 = Produit("CRAY-001", "Crayon HB", "Écriture", 0.15, 0.50, 80, 20) # qte différent

```

```

print(p1 == p2)          # True → même référence
print(sorted([p2, p1]))  # triés par ref
refs_uniques = {p1, p2} # set possible grâce à __hash__
print(len(refs_uniques)) # 1 — ils sont "égaux" pour le set

```

---

## Étape 2 —Gestionnaire de contexte FichierStock (15 min)

Un **gestionnaire de contexte** (with) garantit que des ressources sont toujours libérées, même en cas d'exception.

*# services/stock\_service.py*

```

class FichierStock:

```

```

    """

```

```

    Gestionnaire de contexte pour l'écriture sécurisée du JSON.
    Remplace le write_text() direct de V1 pour garantir la fermeture du fichier.
    """

```

```

    def __init__(self, chemin):
        self.chemin = chemin
        self._fichier = None

```

```

    def __enter__(self):
        """Ouvre le fichier et le retourne pour utilisation dans le bloc with."""
        self._fichier = open(self.chemin, "w", encoding="utf-8")
        return self._fichier # ← valeur de 'as f' dans 'with ... as f:'

```

```

    def __exit__(self, type_erreur, valeur, traceback):
        """

```

```

        Appelé TOUJOURS à la sortie du bloc with, qu'il y ait une erreur ou non.
        type_erreur, valeur, traceback : informations sur l'exception (None si pas d'e
        Retourner False = ne pas étouffer l'exception.

```

```

        """
        if self._fichier:
            self._fichier.close()
        return False # propager l'exception si elle existe

# Utilisation dans sauvegarder()
def sauvegarder(self) -> None:
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    data = [p.to_dict() for p in self._produits.values()]
    with FichierStock(self._chemin) as f: # __enter__ ouvre, __exit__ ferme
        f.write(json.dumps(data, indent=2, ensure_ascii=False))

```

### Comparaison V1 vs V2 :

```

# V1 : risque de fichier non fermé si erreur
self._chemin.write_text(json.dumps(data), encoding="utf-8")

# V2 : fermeture garantie même si json.dumps() lève une exception
with FichierStock(self._chemin) as f:
    f.write(json.dumps(data, indent=2, ensure_ascii=False))

```

## Étape 3 —Méthodes magiques sur StockService (10 min)

# services/stock\_service.py — ajouts V2

```

class StockService:
    # ... (code V1 conservé) ...

    def __len__(self) -> int:
        """len(stock) → nombre de produits en catalogue."""
        return len(self._produits)

    def __contains__(self, ref: str) -> bool:
        """"CRAY-001" in stock → True si le produit existe."""
        return ref in self._produits

    def __iter__(self):
        """for produit in stock: ... → itère sur tous les produits."""
        return iter(self._produits.values())

    def __str__(self) -> str:
        """print(stock) affiche un résumé propre."""
        alertes = len(self._produits_en_alerte())
        return f"Al Qalam Stock · {len(self)} produits · {alertes} alerte(s)"

# Démonstration des méthodes magiques

```

```

stock = StockService()

print(len(stock))           # 10
print("CRAY-001" in stock)  # True
print("XXXX-999" in stock)  # False

for produit in stock:       # __iter__
    if produit.est_en_alerte():
        print(produit)      # __str__ de Produit

# Tri avec __lt__ de Produit
produits_tries = sorted(stock) # stock est itérable → sorted() fonctionne

```

---

#### Étape 4 — Tri par colonne dans le tableau (15 min)

Cliquer sur l'en-tête d'une colonne trie le tableau dans l'ordre croissant / décroissant.

*# ui/frames/stock\_frame.py*

```

class StockFrame(ctlk.CTkFrame):
    def __init__(self, parent, stock, callbacks):
        super().__init__(parent)
        self._tri_colonne = None
        self._tri_inverse = False
        self._construire_tableau()

    def _construire_tableau(self):
        self._arbre = ttk.Treeview(self, columns=COLONNES, show="headings")
        for col in COLONNES:
            # Chaque en-tête déclenche le tri sur sa colonne
            self._arbre.heading(col, text=col.capitalize(),
                                command=lambda c=col: self._trier_par(c))

    def _trier_par(self, colonne: str) -> None:
        """Trie le tableau par la colonne cliquée, inverse à chaque clic."""
        if colonne == self._tri_colonne:
            self._tri_inverse = not self._tri_inverse
        else:
            self._tri_colonne = colonne
            self._tri_inverse = False
        self.rafraichir() # reconstruit le tableau trié

```

---

## Étape 5 — Fiche détail et modification (15 min)

# ui/frames/dialogs.py

```
class DialogueFicheDetail(ctk.CTkToplevel):
    """Fenêtre modale en lecture seule — double-clic sur un produit."""

    def __init__(self, parent, produit):
        super().__init__(parent)
        self.title(f"Fiche — {produit.nom}")
        self.resizable(False, False)
        self._afficher(produit)

    def _afficher(self, p):
        champs = [
            ("Référence", p.ref),
            ("Nom", p.nom),
            ("Catégorie", p.categorie),
            ("Prix achat", f"{p.prix_achat:.3f} TND"),
            ("Prix vente", f"{p.prix_vente:.3f} TND"),
            ("Marge", f"{p.marge_unitaire():.3f} TND"),
            ("Quantité", str(p.qte)),
            ("Seuil min", str(p.seuil_min)),
            ("Valeur stock", f"{p.valeur_stock():.2f} TND"),
            ("Statut", "⚠ ALERTE" if p.est_en_alerte() else "✅ OK"),
        ]
        for label, valeur in champs:
            ligne = ctk.CTkFrame(self, fg_color="transparent")
            ligne.pack(fill="x", padx=20, pady=2)
            ctk.CTkLabel(ligne, text=label + " :", width=120,
                          anchor="w", font=ctk.CTkFont(weight="bold")).pack(side="left")
            ctk.CTkLabel(ligne, text=valeur, anchor="w").pack(side="left")
```

---

## Critères de validation

- ☐ sorted(stock) trie les produits par référence sans erreur
  - ☐ "CRAY-001" in stock retourne True
  - ☐ len(stock) retourne 10
  - ☐ print(stock) affiche le résumé lisible
  - ☐ Cliquer sur une colonne trie le tableau (clic 2 = ordre inverse)
  - ☐ Double-clic sur une ligne ouvre la fiche détail avec toutes les infos
  - ☐ Bouton "Modifier" permet de changer nom, catégorie, prix, seuil
  - ☐ La sauvegarde JSON fonctionne correctement (context manager)
-

### **Pour aller plus loin**

- Ajouter `__format__` sur `Produit` pour utiliser des f-strings personnalisés :  
`f"{p:court}"`
- Implémenter `__add__` sur `StockService` pour fusionner deux stocks :  
`stock_a + stock_b`
- Utiliser `contextlib.contextmanager` comme alternative décorateur au gestionnaire de contexte

---

*Al Qalam Stock Manager —Formation Python Partie II —Sprint V2*